

Bao AI

Testing, Validation and Problem Resolution Report

How Bao AI was tested, what the testing found, and how each problem was fixed and verified.



ABOUT THIS DOCUMENT

This report documents how Bao AI was tested and what the testing found. It covers the automated test suite (449 tests), the evaluation harnesses and their measured results, the tools that check the code, its dependencies and the machine it runs on, and — the core of the report — a catalogue of the problems found during development: for each, the evidence that exposed it, its cause, the fix, and how the fix is verified. The record spans 36 documented review rounds (August 2026) and 56 commits (17–29 September 2026).

Author	Rinsuna Blessing Mahani
Programme	Bachelor of Science (BSc) in Mathematics, final-year project
Institution	Sefako Makgatho Health Sciences University, South Africa
Date	29 September 2026
Code tested	github.com/RinsunaMahani/bao-ai , branch main, 29 September 2026
Companion document	Bao AI — Technical Documentation

Contents

1 Summary	3
1.1 Verification principles	3
2 How Bao was tested	4
2.1 Environments	5
3 The automated test suite	6
4 Evaluation results	7
4.1 Language identification	7
4.2 Tokenizer parity	7
4.3 Knowledge retrieval	8
4.4 Generation	8
4.5 Speech	9
4.6 Performance	9
4.7 Security	10
5 Problems found and how they were resolved	11
5.1 Language identification	11
5.2 Knowledge retrieval	12
5.3 Answer generation	13
5.4 Speech	14
5.5 User interface	15
5.6 Privacy and security	15
5.7 Build, CI and deployment	16
6 Lessons that shaped the testing	16
7 What testing cannot yet show	17
Appendix A Reproducing the results	17

1 Summary

Bao AI was tested at several levels: an automated suite that runs on every change, evaluation harnesses that measure accuracy on labelled data, checks of the machine a demonstration runs on, static and supply-chain analysis, live verification against the real services, and repeated adversarial review in which every claim a reviewer made was checked against the code before being acted on. The results as of 29 September 2026:

What was tested	Result
Automated test suite (20 files)	449 tests. Python 3.11 in CI: 449 passed. Python 3.14 in CI and locally: 438 passed, 11 skipped (need TensorFlow, which has no 3.14 build)
Continuous integration on main	Passing: lint, tests on 3.11 and 3.14, retrieval evaluation, dependency audit, CodeQL
Language identification, deployed model	20/20 on three independent probe sets; benchmark 5/5 at 97.8% average confidence (keyword matcher: 4/5)
Closely related languages (Sepedi, Sesotho, Setswana)	9/9, including the shared greeting "Dumela"
Single-word greetings	14/15 answered in their own language (was 6/15)
End-to-end language routing, 25 languages	24/25 (Luganda is the documented miss)
Tokenizer against Keras	All cases identical, including rare words; re-checked on every CI run
Knowledge retrieval, 70 labelled questions	Precision 0.880, recall 0.733, F1 0.800; 37 of 40 unrelated questions correctly rejected
Knowledge-base self-retrieval	38/38 rows retrieve themselves
Security	No known vulnerabilities in shipped dependencies; no open CodeQL alerts; secret scanning and push protection on; git history free of keys
Performance	Ready in about 7–9 s (was 22.6 s); 273 MB when ready (was 732 MB); first word of an answer after about 2.7 s (was 21.7 s)
Pre-demonstration check on the presentation laptop	13 of 13 checks pass

1.1 Verification principles

Several of the problems in section 5 were not bugs in the code so much as bugs in how it had been checked. The testing therefore follows five rules, each learned from a specific failure:

1. **Measure before claiming, and state the environment.** A result is only as good as the environment it was measured in; every figure in this report says where it came from.
2. **Test the environment that fails, not the convenient one.** Uninstall the dependency, break the thing on purpose: a missing TensorFlow once made the keyword matcher answer while every report claimed the neural model had.
3. **A skipped test is not a passed test.** Checks that skip are reported as unverified, not as green.
4. **A result must name what actually produced it,** never what was requested.
5. **Every fixed bug gets a test that fails on the old code.** Where practical, the test was run against the broken version first to prove it would have caught the fault.

2 How Bao was tested

Figure 1 shows the layers of verification, from the cheapest and most frequent at the bottom to the most judgement-heavy at the top. Each catches problems the others cannot: the test suite proves the code is correct on any machine, but only the pre-demonstration check proves that this machine has its model files, voices and API key.

Adversarial review 36 rounds; every claim from a reviewer checked against the code before acting	each round
Live verification real Gemini calls, real speech synthesis, the Docker container inspected	at each fix
Machine readiness preflight.py (13 checks), voice and model probes	before a demo
Quality gates gates.py (did anything collapse?), scorecard.py (is it better, with error bars?)	before accepting
Evaluation harnesses labelled query sets, detector benchmark, threshold sweeps, routing probe	on demand
Interface tests the real web page driven by Streamlit AppTest	every push (CI)
Unit and integration tests 449 pytest tests; the whole pipeline end to end, offline	every push (CI)
Static and supply-chain analysis ruff (incl. security rules), CodeQL, pip-audit, Dependabot, secret scanning	every push + weekly

Figure 1. Layers of verification, and when each runs.

Tool	What it answers
pytest tests/ (449 tests)	Does each component, and the whole pipeline end to end, behave as specified? Runs offline, with no API key and no network.
Streamlit AppTest	Does the real page render and behave, including settings surviving button clicks?
evaluate.py, bao/rag_evaluation.py	Retrieval self-consistency, and precision, recall and F1 on 70 labelled questions, with a threshold sweep.
benchmark_bao.py	Detector accuracy and latency; which backend actually ran.
compare_retrievers.py, sweep_gates.py	TF-IDF against semantic embeddings; both retrieval gates swept together (24 combinations).
scripts/probe_language_routing.py	One sentence per language through the whole pipeline: is each of the 25 answered in its own language?
scripts/compare_detectors.py, eval_detector_robustness.py	Detector comparison on held-out text, including corrupted input.
scripts/gates.py	Hard gates: API contracts, runtime branches, retrieval integrity, tokenizer parity, tests, lint. A skip reports UNVERIFIED (exit code 2), not success.
scripts/scorecard.py	Is a change a real improvement? Compares with Wilson confidence intervals, so noise is not mistaken for progress.
scripts/preflight.py	Is this machine ready? 13 checks, including a real synthesis and, optionally, one live Gemini call.

Tool	What it answers
scripts/probe_tts.py, probe_mms_codes.py, verify_pan_african_voices.py	Which voices exist and load, established with authenticated requests rather than assumed.
scripts/check_archive.py, scripts/audit_corpus.py	Does a folder or zip carry a secret? Does a training corpus show signs of being generated?
ruff, CodeQL, pip-audit, Dependabot	Lint (including security and bug rules), static security analysis, known vulnerabilities, dependency updates.

2.1 Environments

Environment	Python	Streamlit	Notes	Suite result
Development laptop, Windows 11	3.14	1.59.2	Full speech stack; no TensorFlow	438 passed, 11 skipped
CI, Linux (GitHub Actions)	3.11	1.64	TensorFlow 2.21 for tokenizer parity; PDF support; no speech	449 passed
CI, Linux (GitHub Actions)	3.14	1.64	No TensorFlow	438 passed, 11 skipped
Hardened Docker image	3.11.16	1.64.0	Runs as an unprivileged user; no speech	UI tests 16/16, three consecutive runs

3 The automated test suite

The suite has 449 tests in 20 files. Test names state the behaviour they protect (`test_uploaded_documents_do_not_leak_between_sessions`), and many carry in their docstring the incident that made them necessary, so that a future reader who is tempted to "simplify" the code meets the reason first.

File	Tests	What it checks
<code>test_orchestrator.py</code>	72	The pipeline end to end, offline: blocked input, knowledge-base answers, offline fallbacks, document context, streaming, translation caching, overrides and named languages, the confidence floor, which voice speaks, backup-model reporting
<code>test_voice_coverage.py</code>	48	Which languages have which voice; fallback tiers; substitutions rejected on linguistic grounds; reported coverage matching what synthesis will do
<code>test_speech.py</code>	43	Backend routing, text cleaning and chunking, error explanations, letter folding for the VITS model, safetensors, one load per model, the pinned revision, the warm-up, time limits, log privacy, lazy imports
<code>test_security.py</code>	39	Input validation; disguised injections; ordinary questions that must pass; invisible-character stripping; markdown-image neutralisation; the ReDoS timing guard; name escaping
<code>test_generation_retry.py</code>	39	Transient versus permanent errors, retries, provider-named waits, streaming rules, the backup model, per-thread tracking, reasoning level, output cap, timeout, no function calling
<code>test_pan_african.py</code>	37	The 14-language model: loading, label decoding, version-skew handling, the composite's thresholds, routing probes, Luganda pinned as a known miss
<code>test_knowledge.py</code>	28	Retrieval, stop words, false-positive regressions, the document store (replace, ceiling, clear), unreadable and oversized uploads, PDF page limit, hidden characters
<code>test_tokenizer_parity.py</code>	18	The reimplemented tokenizer against Keras (with TensorFlow), padding and truncation, out-of-vocabulary rules, the trimmed vocabulary
<code>test_retrieval_coverage.py</code>	18	The IDF-weighted coverage gate and the target-language rule
<code>test_streamlit_app.py</code>	16	The real page: renders, answers a greeting in its language, keeps settings across both buttons, survives a missing logo, the Reply in picker and the badge
<code>test_greetings.py</code>	15	Greetings written in-language, one per language, the "Dumela" disambiguation, verbatim serving, counterparts, unreviewed translations never served
<code>test_language_detector.py</code>	15	The keyword and LSTM detectors, class order, padding, the Sotho-Tswana group, pairing, concurrency, and that logs name the detector that answered
<code>test_bootstrap.py</code>	12	One composition root; per-session isolation of memory, documents and the language toggle; shared expensive components
<code>test_rag_evaluation.py</code>	9	The evaluation metrics, and the documented trade-offs pinned
<code>test_config.py</code>	9	Configuration and code agree; the Python floor; the Docker image contains every runtime file; server security settings
<code>test_check_archive.py</code>	9	The secrets scanner: no false alarm on public certificates; private keys and pasted keys found
<code>test_memory.py</code>	8	The rolling window and the per-turn size cap
<code>test_prompt_injection.py</code>	6	The untrusted-document fence and delimiter neutralisation
<code>test_evaluation.py</code>	5	The evaluation harness itself: every row counted, misses explained
<code>test_docs_match_code.py</code>	3	Figures in the documents match the data; every README command exists

4 Evaluation results

4.1 Language identification

Evaluation	Result	Notes
Deployed TFLite model: one formal sentence per language	11/11	0.97–1.00 confidence
Deployed model: short informal greetings	4/4	Different register from the training news text
Deployed model: benchmark queries	5/5	Keyword matcher 4/5 on the same set
Sepedi, Sesotho, Setswana (three sentences each)	9/9	"Dumela" separated at 0.993
Single-word greetings	6/15 → 14/15	After pairing with the keyword matcher; "Dumela" alone cannot be resolved by any detector
25 languages, one sentence each, end to end	24/25	Was 18/25 before the pan-African model was always consulted
Detection latency	1.36 ms mean, 1.55 ms p95	benchmark_bao.py

The confidence floor of 0.50 came from live observation. In the first session with the classifier running, every detection at or above 64% chose the right reply language and every misfire sat between 35% and 42%:

Confidence	Query	Detected	Right reply language?
35%	vuxeni	siSwati	No (Xitsonga)
37%	explain calu=culus in ndebele	English	Yes
42%	what is car in xhosa	Afrikaans	No — answered in Afrikaans
64%	what is car in swati	English	Yes
72%	what is car in ndebele	English	Yes
83%	explain calculus in xitsonga	English	Yes
91%	explain maths in zulu	English	Yes

The pan-African thresholds were measured the same way: that model never exceeded 44% on 24 South African inputs and scored 43–89% where it was right. These are one sentence per language, enough to choose between thresholds, not a calibration claim.

4.2 Tokenizer parity

The app ships a reimplement of Keras's tokenizer so that it never needs TensorFlow. If the two disagreed, the deployed accuracy would silently differ from the trained accuracy. The parity test skipped everywhere until 27 September 2026, when it was first run against TensorFlow 2.21 and Keras 3.15: all ten cases matched exactly. It now runs on every push in CI's Python 3.11 job, and since 29 September it also checks words ranked beyond the vocabulary limit, which is what makes trimming the vocabulary safe.

4.3 Knowledge retrieval

The labelled set has 70 questions: 15 copied from the knowledge base, 15 paraphrases, and 40 unrelated questions, including lexical traps ("what is the capital of Australia") and South African questions the knowledge base does not cover ("who is the current president of South Africa"). It was measured repeatedly as the system changed:

When	System	Exact	Paraphrase	Unrelated rejected	Precision	F1
Round 7	Similarity 0.15 (old default), 20 negatives	15/15	12/15	2/20		
Round 7	Similarity 0.25, 20 negatives	15/15	11/15	8/20		0.765
Round 8	Similarity 0.25, 40 harder negatives	15/15	11/15	8/40	0.433	0.591
Round 13	Coverage gate added (0.70)				0.826	0.731
29 Sep 2026	Shipped: similarity 0.25 and coverage 0.70	15/15	7/15	37/40	0.880	0.800

The sweep of both gates together (24 combinations) gives F1 between 0.687 and 0.821. The best point beats the shipped setting by one paraphrase question, measured on the set the thresholds were tuned on, so the default was not moved. The gates trade against each other: every setting that recovers paraphrases gives back rejection of unrelated questions.

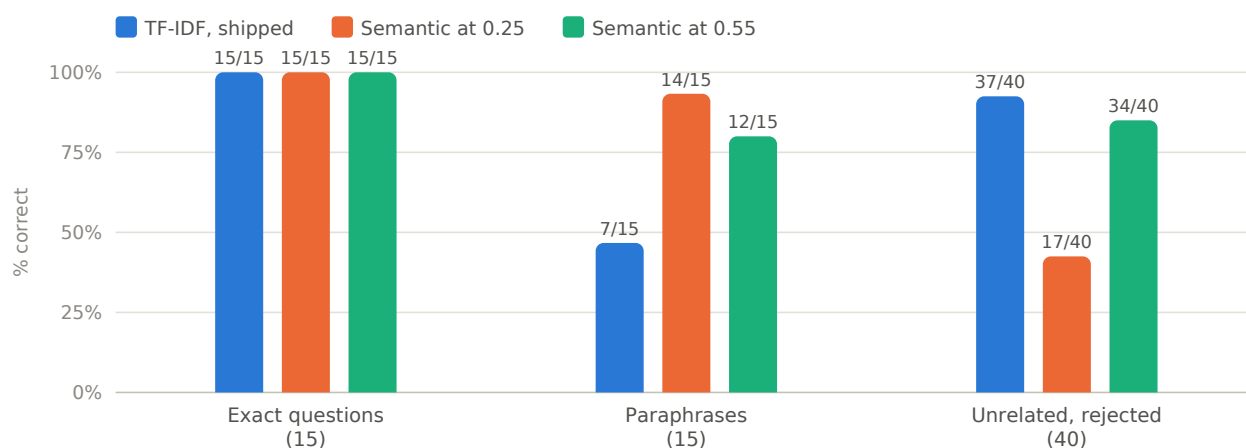


Figure 2. TF-IDF against paraphrase-multilingual-MiniLM-L12-v2, 27 September 2026.

The semantic model was not adopted. The rule, set before the experiment, was better paraphrase recall without losing rejection of unrelated questions; no threshold meets it. Its best F1 (0.857) comes from answering three more unrelated questions as verified facts, and its best threshold is fragile (0.50 to 0.60 moves paraphrase recall from 14/15 to 7/15). A one-off probe combining its similarity with TF-IDF's coverage gate met the rule by one question in each category, which on a development set of this size is noise.

4.4 Generation

Measurement	Result
Availability, 10 calls per model, 19 Sep 2026	gemini-3.5-flash 6 successful and no 503 errors; 3.7-flash 2 and 4; 3.8-flash 1 and 5; 3.6-flash daily quota (20) exhausted
Time to first word, reasoning MINIMAL / default / LOW (API)	6.6 s / 11.9 s / 15.0 s; totals 12.8 / 13.0 / 19.2 s; all answers in correct Xitsonga
Time to first word in the application	21.7 s → 2.7 s

Measurement	Result
Retry behaviour, verified live	A turn that met two consecutive 503 errors answered on the third attempt; a rate-limited turn showed a correctly decreasing wait
Backup model, verified live (28 Sep 2026)	gemini-3.5-flash-lite answered with the project's key
Requested-language compliance	"explain calculus in xitsonga", "explain gravity in Afrikaans" and an isiZulu question each answered in the requested language

4.5 Speech

Measurement	Result
Meta MMS voices for the 11 South African languages (authenticated probe, 30 Aug 2026)	Only Xitsonga and English have one; the other nine return 404 (isiZulu and Afrikaans are covered by Microsoft voices instead)
Alternatives probed	Google Translate speech: HTTP 400 for all nine indigenous languages. Microsoft: 322 voices, 11 relevant locales. One permissive alternative: 6.63 GB
South African VITS model, end to end (18 Sep 2026)	All 11 South African languages produce audio
Pan-African voices (probed 18 Sep 2026)	12 of 14 exist (Oromo as orm, Swahili as swh); Igbo and Lingala have none
Synthesis time against reply length (Xitsonga)	300 characters: 12 s; 400: 15 s; 600: 22 s; a full long reply: 60 s for 152 s of audio. Replies are nevertheless read in full
Speech recognition language	isiZulu as en-ZA: not understood; as zu-ZA: transcribed. The same for Afrikaans as af-ZA
Offline synthesis after the performance changes (29 Sep 2026)	Xitsonga (MMS, safetensors) and Sesotho (VITS, pinned revision) both spoke; no spoken text in the log

4.6 Performance

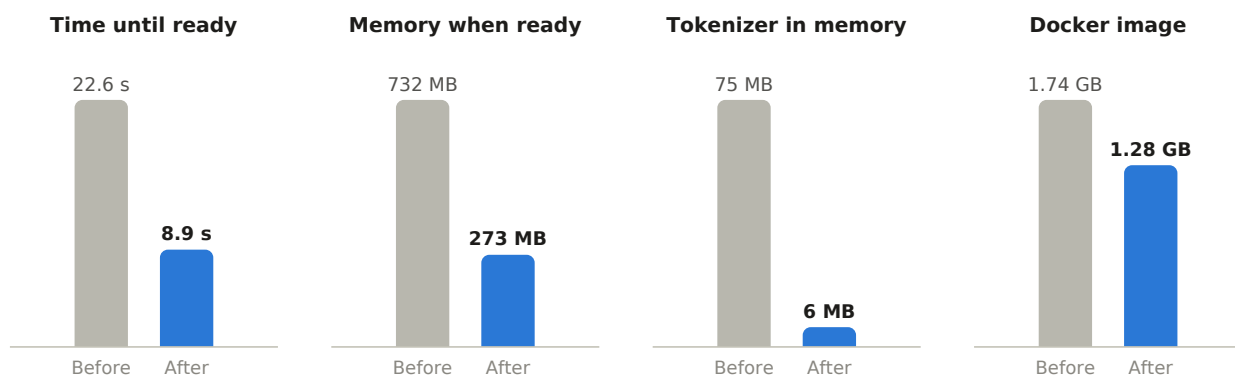


Figure 3. Before and after the optimisation pass of 29 September 2026.

Concurrency was tested directly: 120 simultaneous detections across three languages, all correct, after a lock was added (section 5.1, A16). A slow stub synthesiser confirmed that text appears before audio: 2.4 ms to text, 2,003 ms to audio, where text had previously waited 2,004 ms for the voice.

4.7 Security

Check	Result
pip-audit on what ships (core, ML and PDF requirements)	No known vulnerabilities
pip-audit on the optional speech stack	8 advisories in transformers 4.57.6, all fixed only in 5.x (which coqui-tts cannot use); none reaches a code path Bao uses (analysed in SECURITY.md)
CodeQL, security-extended queries	One alert on first run (ReDoS, see F11), fixed; none open
ReDoS fix, measured on 16,000 crafted repetitions	6.30 s → 0.08 s (linear instead of quadratic)
Git history, all branches	No Google API keys, Hugging Face tokens, private keys or .env files
Hardened container, inspected	Runs as uid 10001; cannot write its code or mounts; CapEff 0; NoNewPrivs 1; port bound to 127.0.0.1
Web server binding	Answers on localhost; a connection through a network address is refused

5 Problems found and how they were resolved

Each entry records what was observed and how, its cause, and the fix with how it is verified. Entries are grouped by area and numbered for reference; dates are given where a problem came from a live session. Almost every fix is pinned by a named test that fails on the old behaviour.

5.1 Language identification

#	Problem, and how it was found	Cause	Resolution, and how it is verified
A1	Wrong input encoding. The classifier received character codes and gave confident, meaningless output; nothing crashed. Found by reading the model's own tokenizer file: word-level, 25,000 words.	The code assumed a character-level model.	Keras's word tokenizer reimplemented from the vocabulary file. Verified identical to the real Keras tokenizer.
A2	Wrong padding. After A1 a spot-check still scored 1/4. On the real model, padding before the text gave near-uniform ~30% outputs; after it, 97-100%.	Keras's default had been assumed.	Post-padding, pinned by a test.
A3	Wrong class order. Outputs mapped to the wrong names. Found by feeding one clearly monolingual sentence per language and recording which output fired.	The interface's language order was used instead of the training order (alphabetical).	TFLITE_CLASS_ORDER, pinned by a test; 20/20 on three probe sets.
A4	A fallback posing as the model, in three places. With TensorFlow missing or broken, the keyword matcher answered while the tests, <code>evaluate.py</code> and the benchmark reported model results; a reviewer reported a model failure that was not real.	Each reported the backend requested, not the one that ran; a test checked only that files existed.	Results carry the backend that ran; tests build the detector and assert backend == "tflite"; scripts warn on fallback. Verified by uninstalling TensorFlow on purpose.
A5	Keras fallback labelled TFLite , which would have satisfied A4's checks falsely.	A hard-coded label.	Label taken from the engine that ran; two tests.
A6	Classifier uninstallable on the presentation laptop. TensorFlow has no Python 3.14 wheels, so the app silently ran on keywords only.	Full TensorFlow required for one interpreter class.	Google's standalone LiteRT runtime (21 MB) preferred. The badge read "ML model" for the first time.
A7	"The input must be in English." Every single-word greeting came back as siSwati at 35% and was answered in English (19 Sep).	Greetings are out of the news vocabulary, so the model returns its prior; keywords were only a load-failure fallback.	LSTM paired with the keyword matcher, which can never overrule a confident model. 6/15 → 14/15; sentence routing unchanged.
A8	Weak guesses chose the reply language. "what is car in xhosa": Afrikaans at 42%, answered in Afrikaans (30 Aug).	Detection drove generation regardless of confidence.	A 0.50 floor: weak detections are shown but not acted on. That sentence is now also recognised as asking for isiXhosa (A14).
A9	A half-applied fix. The badge said "too unsure to use" while the siSwati row was served.	The floor reached generation but not retrieval.	Retrieval uses the reply language too.
A10	Correct detections discarded. "sannu" (Hausa, 43%) answered in English.	Two thresholds calibrated on different models left a dead band between 0.35 and 0.50.	Detections the composite has calibrated are not re-judged; both directions tested.

#	Problem, and how it was found	Cause	Resolution, and how it is verified
A11	Seven pan-African languages never detected. Routing probe: 18/25; Luganda read as Xitsonga at 100%, Hausa as English at 97%.	The pan-African model was asked only when the primary was unsure; an 11-way softmax is confidently wrong outside its languages.	Always consulted; 0.50 or more wins (from measured ranges). 24/25; Luganda pinned as a known miss.
A12	Languages called "0" and "7", then "swa". The prompt even asked for replies in "swa".	Encoded labels, then ISO codes, leaking past the detector.	Labels decoded through the encoder, codes mapped to names; the loader rejects digit labels; tests use the bundle's real labels.
A13	"Interested in French cuisine" switched the reply to French. 5 of 5 such sentences misread.	Any language name after "in" counted as a request.	A request must end the phrase; 19/19 cases correct; retrieval metrics unchanged.
A14	"...in zulu" ignored (28 Sep).	Only official names were recognised.	Everyday names added; "venda" excluded as a place name; tested both ways.
A15	The asked-for language ignored. On flash-lite, "explain gravity in Afrikaans" came back in English twice.	The system instruction's detected language outranked the request inside the sentence.	A named language outranks detection, and (since 28 Sep) the picker, for that message.
A16	Concurrent sessions crashed the classifier. 2 of 3 simultaneous threads died.	TFLite inference is not re-entrant, and the detector is shared.	Inference serialised, output copied inside the lock; 120 concurrent detections correct.
A17	75 MB of unusable vocabulary in memory.	All 679,385 entries kept; only 25,000 can reach the model.	Trimmed at load to 6 MB; parity with Keras, rare words included.
A18	A log line credited the wrong detector (found while writing this report): the terminal read "pan-African model says Xitsonga" for "avuxeni", which the keyword matcher had answered.	The message named one detector, whichever the composite wrapped.	Log lines name the backend that actually answered; a test pins the wording.

5.2 Knowledge retrieval

#	Problem, and how it was found	Cause	Resolution, and how it is verified
B1	Answers matched on function words. "what is quantum physics" matched the currency fact at 0.352 on "what is"; "speed of light" at 0.532.	Stop words weighted; a later rewrite bypassed the stop-word list and reintroduced the bug.	Curated stop-word list, enforced in one place with a warning comment; now 0.000; pinned by a regression test.
B2	Confident wrong "verified" answers. "minimum wage in south africa" → currency fact at 0.666; population (0.666) and voter registration (0.460) likewise.	TF-IDF drops unknown words, so the question was scored as "south africa".	IDF-weighted coverage gate at 0.70, weighting and threshold chosen by measurement: false positives 22 → 4.
B3	No threshold fits both paraphrases and unrelated questions. At the old 0.15, 18 of 20 unrelated questions got a knowledge-base answer.	TF-IDF measures word overlap, not meaning.	Default 0.25; both gates swept together (tuning exhausted); semantic embeddings measured and not adopted under a rule set in advance.
B4	"calculus in xitsonga" served a tutorial-venue row (similarity 0.303, coverage 100%).	Known words in an unknown combination: the opposite of B2.	Questions naming a target language go to generation; tested both ways.
B5	Badge and answer disagreed. "dumela": badge Sesotho, answer from the Setswana row.	Retrieval never saw the detector's answer; distinct triggers had hidden the ambiguity.	All three rows answer to "dumela"; the reply language chooses among them.

#	Problem, and how it was found	Cause	Resolution, and how it is verified
B6	Curated answers corrupted by translation. A Setswana greeting returned with quote marks, a "me/thusa" artefact and a note on Lesotho spelling, all read aloud.	Non-English curated rows were sent to the translator.	Curated South African-language rows served verbatim; English facts still translated; both pinned.
B7	One table, two systems. The per-category results scored similarity alone while the F1 beside them included the gate, so the wrong weakness was reported.	Two functions with different defaults.	Same default; both views pinned separately; README rewritten.
B8	The evaluation harness hid a miss. A row whose retrieval returned nothing lowered accuracy without appearing as a miss.	The module whose output becomes report claims had 0% test coverage.	Counted as a miss; harness tests added.
B9	Wrong-language answer under the picker (28 Sep): with Sesotho chosen, "avuxeni" returned the Xitsonga greeting under "Replying in Sesotho".	The chosen language was preferred only among rows that matched.	The same fact's row in the chosen language (shared Canonical_Id, or that language's greeting); an honest badge when none exists; page and unit tests.
B10	Unreviewed translations of emergency facts must never be served as verified (14 drafts).	Prevention, not an incident.	Excluded from the index; the counterpart lookup cannot bypass the exclusion (tested).

5.3 Answer generation

#	Problem, and how it was found	Cause	Resolution, and how it is verified
C1	One "high demand" 503 lost the turn, although the same request succeeded moments later (19 Sep).	Transient errors were treated as fatal.	Retries for 429, 5xx and network errors, read from the status code; verified live.
C2	Retries deepened rate limits. The free tier allowed 5 requests a minute and asked for a 13-second wait.	Three sub-second retries spent more of the same quota.	Waits over 2 s are not retried; the user sees the wait ("about 13 seconds").
C3	A retried stream would duplicate text on screen.	Chunks are displayed as they arrive.	No retry once text has appeared; tested.
C4	The model was chosen by recency. 3.6-flash allowed 20 requests a day; 3.7 and 3.8 were mostly busy.	Assumption instead of measurement.	Ten calls per model measured; gemini-3.5-flash chosen; quotas are per model.
C5	22 seconds before the first word.	Gemini 3.x reasons before it streams.	Reasoning set to MINIMAL: 2.7 s, with the same answers.
C6	Memory re-sent 12,656 characters (about 3,200 tokens) on every request after six long exchanges.	Turns were counted, not sized.	400 characters per turn, the cut marked: about 2,000 characters for six.
C7	Busy through all retries (28 Sep).	One model, one queue.	Backup model before the busy message, tracked per thread, named on the badge; verified live.
C8	No request timeout and no output limit.	SDK defaults.	60 s timeout (also sent to the server), 4,096-token cap, function calling off; tested.
C9	Identical translations paid for repeatedly; then an unbounded cache.	No cache, then no bound.	LRU cache of 512; failures never cached; tested.

5.4 Speech

#	Problem, and how it was found	Cause	Resolution, and how it is verified
D1	Wrong voice for cross-lingual replies. An English question answered in Xitsonga was read by an English voice.	The voice followed the question.	The voice follows the reply.
D2	Markdown read aloud. Asterisks spoken in English, Afrikaans and isiZulu replies.	Cleaning lived inside a step only the MMS backend runs.	Cleaning for every backend; apostrophes kept because they are spelling; 9 tests.
D3	A missing voice re-downloaded on every message: 5 messages, 5 download attempts.	functools.cache does not cache exceptions.	Successes and failures remembered; a reset function; tested.
D4	A confident wrong diagnosis: "mms-tts-sot does not exist" was concluded from a 401 (unauthenticated), not a 404.	The library prints one message for 401 and 404; the diagnosis matched text.	Branch on the status code; ambiguous cases say so; an authenticated probe confirmed the real 404s.
D5	English lost its offline voice.	English was mapped to mms-tts-afr, which does not exist.	Mapped to eng; code default and config tested together.
D6	Seven of eleven languages had no voice.	No open voice exists in MMS, Google or Microsoft (probed).	South African VITS model, opt-in for licence reasons: 11/11 produce audio.
D7	That model would not load, then dropped letters ("thušo" spoken as "thuo").	The trainer's machine paths in its config; dependency conflicts; a 138-symbol alphabet.	Patched private copy of the config; dependency caps; letters folded to the nearest equivalents.
D8	Pan-African replies were silent.	No voice codes mapped for the 14.	12 verified MMS codes (Oromo is orm, not gaz) and 4 Microsoft voices; Igbo and Lingala say so.
D9	A quoted language stole the voice: a Xitsonga reply quoting the Shona anthem was read by a Shona voice (20 Sep).	The whole reply was sampled (Shona 85%); its opening says Xitsonga (100%).	Voice detected from the reply's opening.
D10	Half the answer read aloud.	A 400-character cap chosen for speed.	Full replies by default: for someone who cannot read the screen, the voice is the answer.
D11	The chosen language's voice reading other text (28 Sep): a Xitsonga greeting and the English busy message read by the Sesotho voice.	The choice was treated as the language of the words.	Results carry the known text language, and the voice follows it; tested.
D12	Speech recognised as the wrong language: the first spoken message was always treated as English. isiZulu as en-ZA failed; as zu-ZA, transcribed.	The recogniser needs the language in advance; the previous turn's was used.	"I'll speak in" setting, shared by web and console; failures name the language.
D13	Twenty seconds before the page appeared. Importing Bao took 19.8 s and 574 MB.	The voice stack was imported at start-up.	Lazy imports and a background warm-up: about 5 s; a test checks torch is not imported.
D14	Reply text in the terminal log. Coqui logged each spoken sentence.	A global logging configuration let every library's INFO through (and doubled every line).	Module logger; third-party loggers held at WARNING, including after lazy imports; tested.
D15	A voice choice read from the wrong turn. An explicit language choice was replaced by a guess after an intervening turn.	The choice was stored on the shared orchestrator, but speech runs later.	Moved onto the per-turn result; tested with interleaved turns.

5.5 User interface

#	Problem, and how it was found	Cause	Resolution, and how it is verified
E1	"83% — too unsure to use" on the showcase query (23 Sep).	One explanation for two opposite reasons.	"You asked for X" distinguished from "too unsure".
E2	"New conversation" reset the visitor's settings. Found by driving the real page with AppTest; no test had rendered it before.	<code>st.rerun()</code> inside a handler stopped the run before later widgets were drawn (Streamlit 1.59).	<code>on_click</code> callbacks; the page test fails on the old code.
E3	The Docker image could not render the page.	<code>.dockerignore</code> excluded the logo's folder.	Logo shipped; a missing logo degrades to an icon; a test fails if any runtime file is excluded.
E4	A latency label contradicted its breakdown.	It measured text, while the stages included speech.	"Time to answer", with the voice as its own figure.
E5	A bad PDF replaced the page with a traceback.	The PDF library raises several error types; nothing caught them.	Unreadable files return no text and are named; 7 malformed-file cases tested.
E6	Uploads invisible, irreversible, and duplicated on re-index.	No listing, no clear button; every click appended.	Listing, clear button, replace-on-reindex, a 400-chunk ceiling.

5.6 Privacy and security

#	Problem, and how it was found	Cause	Resolution, and how it is verified
F1	One visitor's conversation and files reached the next. A second session's three differently worded questions all retrieved the first session's private file (20 Sep).	<code>st.cache_resource</code> is shared by all sessions.	Per-session memory and documents; isolation tests.
F2	Live API keys inside a shared zip.	<code>.gitignore</code> and <code>.dockerignore</code> do not govern a hand-built archive.	<code>.env</code> removed; <code>scripts/check_archive.py</code> refuses secrets by name and by content; rotation left to the owner.
F3	User questions written to logs on three error paths.	Query text in log messages.	Only lengths logged.
F4	Uploaded documents could inject instructions.	Document text was pasted under a plain heading.	Untrusted-data fence, neutralised delimiters, a system rule; 6 tests.
F5	Conversation could leak through a markdown image, fetched by the browser without a click.	Streamlit renders markdown images.	Images turned into links in every rendered answer, excerpt and message.
F6	Disguised injections passed screening.	Exact-phrase matching.	Normalisation and stripping of invisible characters; 12 attack and 10 ordinary-question tests.
F7	The app and its API key were on the campus network ("Network URL" in the server log).	Streamlit's defaults: every interface, tracebacks shown, statistics sent.	Localhost only, no tracebacks, no statistics, 10 MB uploads; network access verified refused.
F8	Unbounded uploads: a 200 MB file would be decoded and fully chunked.	No limits before chunking.	10 MB, 300 pages, 2,000,000 characters, cut before chunking; tested with real PDFs.
F9	Container ran as root, with a compiler, on every interface.	Default image practice.	Unprivileged user, no toolchain, pinned digest, read-only mounts, capabilities dropped; inspected.
F10	Unpinned supply chain.	Latest-revision model download; movable action tags; no audits.	Pinned revision, safetensors, SHA-pinned actions, pip-audit, CodeQL, Dependabot, private vulnerability reporting.

#	Problem, and how it was found	Cause	Resolution, and how it is verified
F11	ReDoS in the new image filter , found by CodeQL's first run: 6.3 s on crafted text.	Unbounded repetition made each "![\" scan to the end.	Bounded pattern (0.08 s on the same input); timing test.
F12	Dependabot's first proposals would have broken the voice (transformers ≥ 5.17) and changed the image's Python; CI passed because it does not install speech.	Default "raise every minimum" strategy.	increase-if-necessary, deliberate caps ignored; both proposals closed with explanations.

5.7 Build, CI and deployment

#	Problem, and how it was found	Cause	Resolution, and how it is verified
G1	CI never loaded the model : green builds, every TFLite test skipped.	Model files arrived as Git LFS pointer stubs.	The two runtime model files fetched from LFS (27 MB, not 96 MB).
G2	A skip reported as a pass : tokenizer parity skipped wherever TensorFlow was absent.	Skips look green.	gates.py reports UNVERIFIED; CI's 3.11 job installs TensorFlow, so parity runs on every push.
G3	Python 3.10 declared, 3.11 needed . On 3.10 the configuration was silently ignored and an older scikit-learn was installed.	tomllib and scikit-learn 1.8+ need 3.11 (seen in CI's install log).	Floor raised to 3.11; CI tests 3.11 and 3.14; a test holds pyproject, CI and README together.
G4	Pickle version skew : 245 of 280 test warnings.	Model saved under scikit-learn 1.8, loaded under 1.9.	Re-exported and verified bit-identical over 25 inputs; pinned to 1.9.x; only minor mismatches warn.
G5	The Docker image under-delivered : keyword matcher only, no PDFs, and no start without .env.	Only core requirements installed; a required env file.	ML and PDF extras; optional env file; verified in the container.
G6	Two dependency manifests disagreed .	Drift between pyproject and requirements files.	Aligned; transformers capped below 5.
G7	Tests that would fail in CI . Found by running a CI-like environment first: path resolution and disabled widgets differ between Streamlit 1.59 and 1.64.	Environment differences.	Absolute paths, declared preconditions: 350 $\rightarrow$ today 449 passing in CI.
G8	A race in the UI tests inside Docker .	Tests faked installed voice backends; the warm-up tried the real import.	Warm-up stubbed in the fixture; three consecutive container runs pass.
G9	Documents drifted from the code .	Counts and thresholds written by hand in several places.	Tests check document figures against the data; test counts not hard-coded.
G10	Tooling mismatch : the pre-commit formatter would have rewritten 53 of 79 files.	Hook and CI configured differently.	Hook matches CI; ruff bounded to one series.

6 Lessons that shaped the testing

- **Check what actually ran**. The same masking bug appeared in three places (A4); each time it produced plausible wrong output rather than an error.
- **A confidence belongs to the model that produced it**. Comparing numbers across models caused A10 and hid the languages in A11.
- **Per-turn facts belong on the per-turn object** (D15): when work is split so part runs later, the state it reads must move with it.
- **An error message is evidence of what a library printed, not of what happened** (D4).

- **Test a checker against data it should pass, not only data it should fail:** the secrets scanner first failed every machine with a public certificate bundle, and a check that always fails gets ignored.
- **A partial fix can be worse than none** (A9): the interface then contradicts itself.
- **Do not write the test you are graded on.** The held-out retrieval set ships empty on purpose, to be written by someone who is not tuning the thresholds; generated evaluation sentences in languages the author does not speak were declined for the same reason.

7 What testing cannot yet show

- **The 0.98 macro F1 is the training evaluation.** Its 63,613-sentence test set is not available; what was validated here is the deployed inference path (20/20, and tokenizer parity).
- **Retrieval thresholds were tuned on the evaluation set,** so its figures are development figures. `eval/rag_holdout.csv` and `scripts/holdout_eval.py` exist for an independent measurement.
- **Linguistic correctness needs first-language speakers:** the greetings were written with AI assistance and are marked curated, and the 14 translations await review. Automated tests cannot establish either.
- **Voice quality is judged by ear.** The eleven South African voices were listened to and approved by the author; the twelve pan-African voices have not been heard by speakers of those languages.
- **Known misses stay pinned:** Luganda is routed to Xitsonga, and a test asserts it so a future fix is noticed.
- **Prompt injection written in other languages,** or phrased in new ways, is not caught by screening.
- **Load:** concurrency was tested with threads (120 simultaneous detections, concurrent model loads), not with many real users.

Appendix A Reproducing the results

```

pytest tests/ -v
ruff check .
python evaluate.py
python benchmark_bao.py
python sweep_gates.py
python compare_retrievers.py
python scripts/probe_language_routing.py
python scripts/gates.py
python scripts/scorecard.py --save baseline.json
python scripts/preflight.py --online
python scripts/probe_tts.py
python scripts/check_archive.py
pip-audit -r requirements-ml.txt -r requirements-pdf.txt

# the suite (449 tests)
# lint, including security and bug rules
# self-retrieval and the 70-question evaluation
# detector benchmark and latency
# both retrieval gates, 24 combinations
# TF-IDF vs semantic (requirements-semantic.txt)
# 25 languages, end to end
# hard gates; exit 2 means unverified
# then --compare baseline.json after a change
# is this machine ready (one live call)
# which voices load, per language
# secrets in the working tree or a zip
# known vulnerabilities

```

Where the evidence lives

REVIEW.md records the 36 review rounds claim by claim. Commit messages from 17 September 2026 carry the evidence for each later fix. SECURITY.md lists every security control and the accepted risks. The companion Technical Documentation explains the system these tests exercise.